SCALO – Smart Connection & Layover Optimizer

Report di valutazione per la scelta dell'approccio

Abstract

Questo documento analizza tre possibili approcci per l'acquisizione dei dati di volo nel progetto SCALO andando a motivare la scelta dell'approccio basato su **SerpApi Google Flights** come fonte primaria, integrato con l'**API GetYourGuide** per la valorizzazione degli scali. Vengono esaminate in dettaglio le criticità degli approcci alternativi (scraping diretto con Puppeteer e API partner diretti) e illustrato il modello architetturale raccomandato.

Contents

1 Contesto e requisiti del problema **2**

2 Analisi comparativa delle tre strategie **2**

2.1 Strategia A – Web Scraping diretto con Puppeteer 2

2.1.1 Funzionamento tecnico 2

2.1.2 Criticità tecniche 3

2.2 Strategia B – Integrazione diretta con API (Skyscanner, Amadeus, Duffel) . . 3

2.2.1 Analisi dei provider disponibili 3

2.2.2 Vincoli tecnici 4

2.3 Strategia C – SerpApi Google Flights (la nostra proposta) 4

2.3.1 Perché Google Flights? 4

2.3.2 Parametri rilevanti per SCALO 4

2.3.3 Struttura della risposta JSON e logica del filtro scalo 5

2.3.4 Vantaggi tecnici dell'approccio 6

3 Confronto sintetico tra le strategie **6**

4 Integrazione con GetYourGuide: valorizzazione dello scalo **6**

4.1 Razionale e valore aggiunto 6

Contesto e requisiti del problema

Il progetto SCALO richiede la capacità di ricercare itinerari aerei imponendo un vincolo ben preciso: lo *scalo obbligatorio*. Questa funzionalità non è offerta solitamente da nessun motore di ricerca dei voli disponibile al pubblico, il che rende necessario costruire uno strato applicativo in grado di:

- interrogare una fonte dati affidabile e aggiornata con i prezzi reali e acquistabili;
- filtrare o costruire possibili itinerari che includano un aeroporto di scalo specifico (codice IATA);
- estrarre informazioni granulari sullo scalo: aeroporto, durata dello stopover, compagnia aerea per ogni segmento;
- restituire link di acquisto funzionanti verso le piattaforme partner associate;
- tenere in considerazione i nostri vincoli accademici: tempi limitati, zero budget per licenze, codice manutenibile.

Abbiamo pensato a tre strategie tecniche percorribili, che vengono analizzate nelle sezioni seguenti.

Analisi comparativa delle tre strategie

2.1 Strategia A – Web Scraping diretto con Puppeteer

Puppeteer è una libreria Node.js sviluppata dal team di Google Chrome che espone un'API ad alto livello per il controllo programmatico di un browser basato su Chromium. Permette di automatizzare interazioni con pagine web dinamiche, intercettare richieste di rete e simulare il comportamento di un utente reale.

Funzionamento tecnico

L'idea sarebbe quella di andare ad automatizzare la navigazione su siti come Google Flights, Kayak o Skyscanner: il browser viene aperto in modalità headless, vengono compilati i campi del form di ricerca e i risultati vengono estratti dal DOM una volta renderizzati via JavaScript.

Criticità tecniche

Motivi per cui questa strategia non è adatta al nostro progetto:

- **Anti-bot sistematici.** Google Flights, Kayak e le principali OTA (Online Travel Agency) impiegano sistemi di rilevamento avanzati, verifica della latenza degli input, CAPTCHA dinamici (Cloudflare Turnstile, hCaptcha). Andare a raggirare i seguenti sistemi richiede l'uso di plugin stealth (`puppeteer-extra-plugin-stealth`), rotazione di proxy e simulazione di movimenti del mouse, che non è il nostro scopo del progetto.
- **Fragilità strutturale.** Ogni aggiornamento dell'interfaccia del sito target invalida i selettori CSS/XPath utilizzati dallo scraper. Questo porterebbe ad un continuo aggiornamento del codice sorgente della piattaforma rendendolo molto instabile.
- **Violazione dei Terms of Service.** Nonostante il nostro sia un progetto a puro scopo accademico, non potremmo andare legalmente ad effettuare lo scraping diretto sui loro siti principali.
- **Latenza elevata.** L'avvio di un'istanza browser, la navigazione completa e l'attesa del rendering JavaScript introducono latenze non indifferenti, incompatibili con il requisito di risposta entro 3 secondi definito negli obiettivi del progetto.
- **Scalabilità nulla.** Ogni istanza Chromium consuma circa 200–300 MB di RAM. Gestire ricerche concorrenti richiede un'orchestrazione di processi complessa, che rappresenta un overhead architetturale ingiustificato.

2.2 Strategia B – Integrazione diretta con API (Skyscanner, Amadeus, Duffel)

Questa strategia prevede l'integrazione con le API ufficiali dei principali aggregatori di voli.

Analisi dei provider disponibili

Provider	Accesso	Dati scalo	Free Limits
Skyscanner API	Solo partner commerciali approvati	Parziali	Non disponibile
Amadeus	Registrazione richiesta, approvazione manuale	Sì, ma verboso	2 000 richieste/mese
Duffel	Account business	Completo	Limitato in sandbox
Kiwi/Tequila	Registrazione, quota variabile	Sì	500 richieste/giorno

Vincoli tecnici

- **Barriere di accesso.** Skyscanner API è riservata esclusivamente a partner commerciali con un volume di traffico minimo garantito. Non è accessibile per un progetto accademico senza accordi formali.
- **Quote restrittive.** Amadeus offre un piano sandbox con 2 000 richieste/mese, sufficienti per lo sviluppo ma non per test intensivi. Il piano produzione richiede contratto commerciale.
- **Complessità del dato sullo scalo.** Le API come Amadeus restituiscono itinerari multi-segmento in formato JSON annidato (standard IATA/IOTA). Estrarre e normalizzare la struttura degli scali richiede una logica di parsing complessa, specialmente per voli con connessioni multiple o code-share.
- **Assenza del filtro scalo obbligato.** Nessuna delle API analizzate espone un parametro nativo per imporre uno specifico aeroporto di scalo. L'approccio richiederebbe di interrogare tutti gli itinerari e filtrare lato server, con un numero di chiamate API potenzialmente molto elevato.
- **Basso valore differenziante.** Dal punto di vista progettuale, integrare una singola API di voli è un task standard privo di elemento innovativo. Non valorizza le competenze sviluppate né produce un risultato distinguibile.

2.3 Strategia C – SerpApi Google Flights (la nostra proposta)

SerpApi è un servizio di terze parti che espone un'API strutturata per interrogare Google Flights, restituendo i risultati in formato JSON pulito e normalizzato. A differenza dello scraping diretto, è SerpApi a gestire internamente il browser headless, la compliance con Google e il rate limiting, offrendo all'utente una semplice interfaccia REST.

Perché Google Flights?

Google Flights aggrega dati da oltre 300 compagnie aeree e GDS (Global Distribution Systems) in tempo reale. I prezzi sono acquistabili e aggiornati, la copertura globale è la più ampia disponibile gratuitamente, e l'interfaccia struttura già nativamente i voli per segmenti e scali.

Parametri rilevanti per SCALO

L'API SerpApi Google Flights espone parametri che si mappano direttamente sui requisiti del progetto:

`Parametro`	**Utilizzo in SCALO**
`departure_id`	Aeroporto IATA di partenza (es. `MXP`)
`arrival_id`	Aeroporto IATA di destinazione (es. `BKK`)
`type`	`1` = andata/ritorno, `2` = solo andata
`outbound_date`	Data di partenza in formato `YYYY-MM-DD`
`return_date`	Data di ritorno (solo per round trip)
`layover_duration`	Filtro durata scalo in minuti (`min,max`)
`exclude_conns`	Esclude aeroporti di connessione indesiderati
`stops`	Numero di scali (impostare ≥ 1 per garantire almeno uno scalo)
`departure_token`	Token per recuperare i voli di ritorno (round trip)
`booking_token`	Token per recuperare le opzioni di prenotazione acquistabili

Struttura della risposta JSON e logica del filtro scalo

La risposta dell'API include un campo `layovers` per ogni itinerario, contenente le informazioni strutturate di ogni scalo intermedio. Questa è la base su cui costruire il filtro per scalo obbligato lato backend:

Listing 1: Struttura semplificata della risposta SerpApi Google Flights

```
{
  "best_flights": [
    {
      "flights": [
        {
          "departure_airport": { "id": "MXP", "time": "2026-06-10 08:00" },
          "arrival_airport":   { "id": "IST", "time": "2026-06-10 11:45" },
          "airline": "Turkish Airlines",
          "flight_number": "TK1888",
          "duration": 165
        },
        {
          "departure_airport": { "id": "IST", "time": "2026-06-10 14:30" },
          "arrival_airport":   { "id": "BKK", "time": "2026-06-11 03:15" },
          "airline": "Turkish Airlines",
          "flight_number": "TK68",
          "duration": 585
        }
      ],
      "layovers": [
        {
          "id": "IST",
          "name": "Istanbul Airport",
          "duration": 165,
          "overnight": false
        }
      ],
      "total_duration": 915,
      "price": 487,
      "booking_token": "eyJh..."
    }
  ]
}
```

Il backend di SCALO implementa quindi un filtro post-query che verifica, per ogni itinerario restituito, se l'array `layovers` contiene almeno un elemento con `id` uguale allo scalo richiesto dall'utente. Questo approccio è semplice, robusto e completamente disaccoppiato dalla fonte dati.

Listing 2: Pseudocodice del filtro scalo obbligato (Node.js/Express)

```
// adapter/serpapi.js
async function searchFlights({ origin, destination, requiredLayover, date
    }) {
  const params = {
    engine: "google_flights",
    departure_id: origin,
    arrival_id: destination,
    outbound_date: date,
    stops: 2,  // almeno uno scalo
    api_key: process.env.SERPAPI_KEY,
  };
  const raw = await serpapi.search(params);
  const all = [...(raw.best_flights || []), ...(raw.other_flights || [])];

  // Filtro scalo obbligato
```

```
  return all.filter(flight =>
    flight.layovers.some(l => l.id === requiredLayover.toUpperCase())
  );
}
```

Vantaggi tecnici dell'approccio

- **Accesso senza approvazione.** SerpApi è disponibile con registrazione self-service. Il piano gratuito include tutto quello che ci serve.
- **Dati strutturati e normalizzati.** Nessun parsing HTML, nessun selettore CSS fragile. Il JSON restituito ha uno schema stabile e documentato, con tipizzazione prevedibile.
- **Latenza contenuta.** Le chiamate REST a SerpApi hanno una latenza media di 1–4 secondi, compatibile con il requisito di risposta sotto i 3 secondi con caching lato backend.
- **Prezzi e link di acquisto reali.** Ogni risultato include un `booking_token` che permette di recuperare le opzioni di acquisto effettive, garantendo che i link verso i partner siano funzionanti.
- **Architettura modulare.** L'adattatore SerpApi è un modulo isolato. Aggiungere un secondo provider (es. Amadeus) in futuro richiede solo la scrittura di un nuovo adattatore con la stessa interfaccia.

Confronto sintetico tra le strategie

Criterio	Puppeteer	API diretta	SerpApi
Accesso immediato	Sì	Parziale	Sì
Dati strutturati	No	Sì	Sì
Latenza < 5s	No	Sì	Sì
Compliance ToS	No	Sì	Sì
Filtro scalo nativo	No	No	Parziale*
Prezzi acquistabili	Difficile	Sì	Sì
Manutenibilità	Bassa	Alta	Alta
Complessità setup	Alta	Media	Bassa

* Il filtro per scalo specifico viene implementato lato backend su dati già strutturati.

Integrazione con GetYourGuide: valorizzazione dello scalo

4.1 Razionale e valore aggiunto

Il differenziatore principale di SCALO rispetto a qualsiasi motore di ricerca voli esistente non è solo il filtro per scalo obbligato, ma la possibilità di *trasformare lo scalo da vincolo tecnico a esperienza pianificata.* L'utente che sceglie di passare da Istanbul non lo fa solo per risparmiare: vuole esplorare la città. SCALO è l'unico strumento che unisce la ricerca del volo con la pianificazione dell'esperienza nello scalo.